Lab 2 Report — Signal Processing

Yacoub Dawli — yada2100@student.miun.se **22 October 2026**

1. Introduction

In this lab, signals were studied in both the time and frequency domains. Different windows and filters were tested, and my voice recording was used to see how sound changes over time.

2. Experimental Setup

Python with NumPy, SciPy and Matplotlib was used. The main signal was made from three cosine waves at 50, 55 and 120 Hz, sampled at 250 Hz for 0.6 seconds (150 samples). Two digital systems were also tested and filters were designed at a sampling rate of 2,500 Hz. For the voice task, my WAV recording was analyzed.

3. Results and Analysis

3.1 Time-domain analysis

The time plot shows the three waves added together. The FFT shows peaks at 50, 55 and 120 Hz. The full spectrum also has matching peaks at negative frequencies. This is normal for a real cosine signal; the negative side does not mean extra sounds were added.

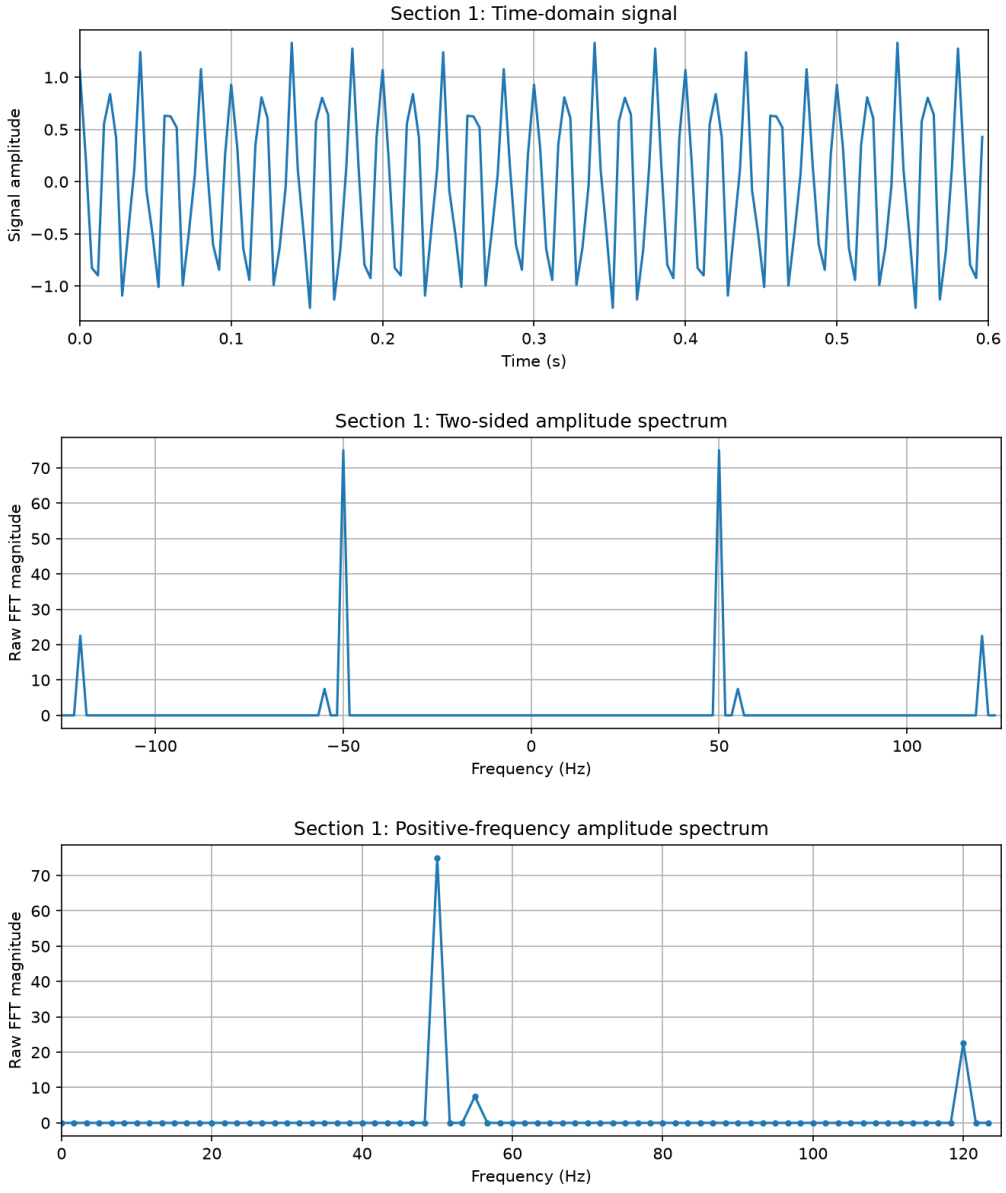


Figure 1. Top: time-domain signal. Middle: two-sided amplitude spectrum. Bottom: positive-frequency amplitude spectrum.

3.2 Frequency identification

Rectangular, Hann and Hamming windows were compared. For this signal, the rectangular window separated 50 and 55 Hz most clearly. Its peaks were narrower. Hann and Hamming made the peaks wider but generally help reduce leakage. The rectangular result is especially clear here because the frequencies fall exactly on FFT frequency points.

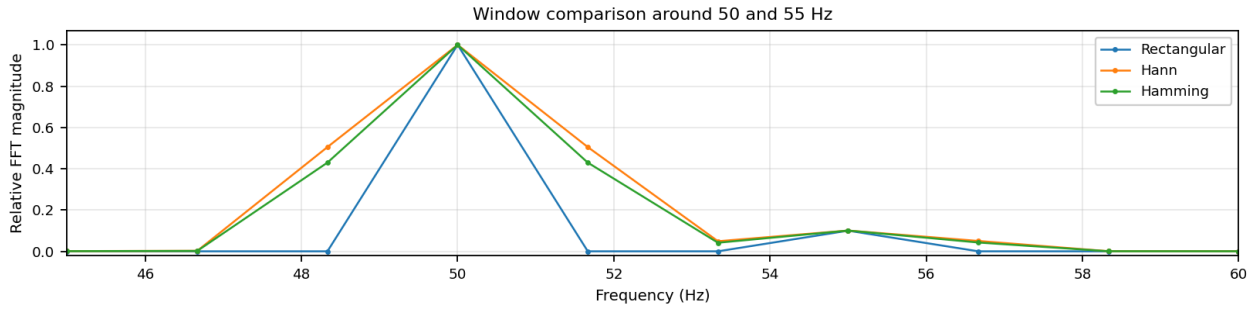


Figure 2. The three windows around the nearby 50 and 55 Hz frequencies.

3.3 Linear time-invariant systems

For System 1, convolution gave the eight values in the given impulse response, followed by zeros. This is an FIR system because its impulse response ends. For System 2, a loop and SciPy's lfilter were used with b = [1, 0.3] and a = [1, 0.6, −0.35]. Both methods gave the same 100 output samples (maximum difference: 0). System 2 is IIR because previous output values are used to calculate the next ones.

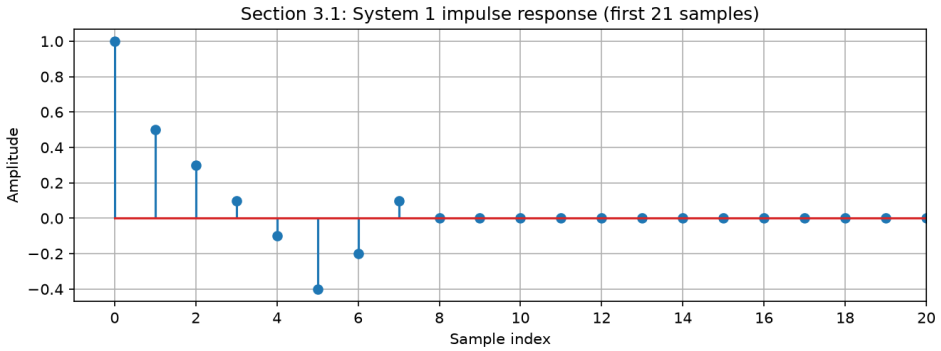


Figure 3. System 1 impulse response: eight nonzero samples, then zeros.

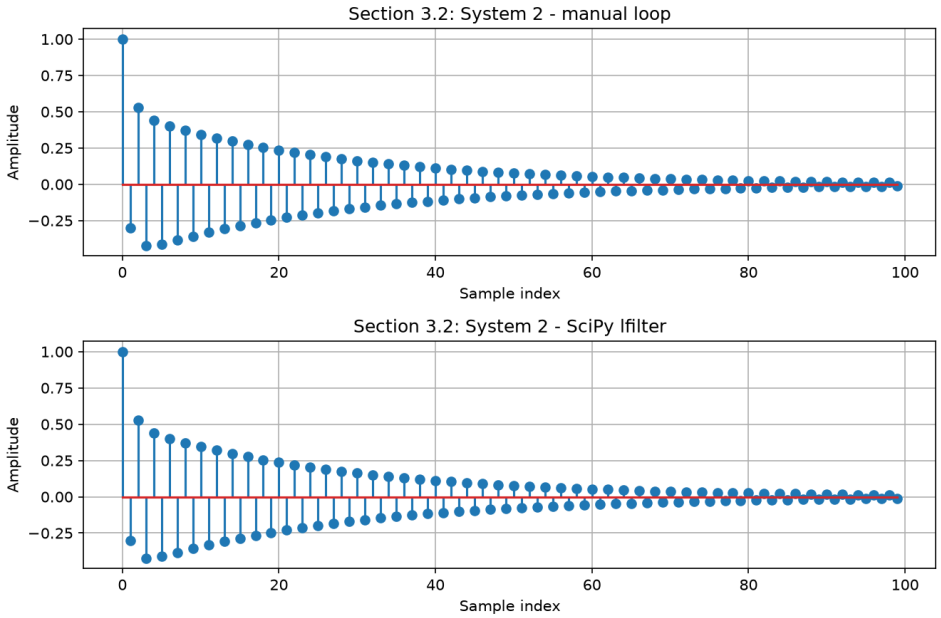


Figure 4. System 2: matching 100-sample responses from the loop and SciPy lfilter.

3.4 Frequency-domain analysis

The frequencies, amplitudes and phases were read from the FFT. After scaling the FFT magnitudes, all three amplitudes and phases agreed with the original signal.

Frequency	Amplitude	Phase
50 Hz	1.0	0 rad
55 Hz	0.1	0.785 rad (π/4)
120 Hz	0.3	1.571 rad (π/2)

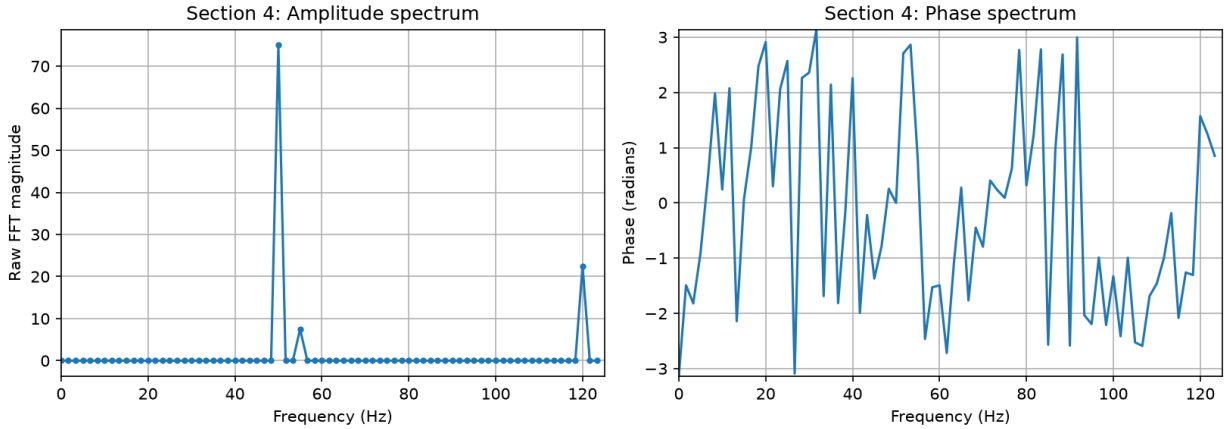


Figure 5. The signal's amplitude spectrum (raw FFT magnitude) and phase spectrum.

3.5 System frequency response

A 50 Hz cosine was passed through System 2. Its amplitude dropped from 1.0 to 0.7481 and its phase shifted to −0.0103 radians. The system response predicted an amplitude of 0.7454 and a phase of −0.0128 radians. The amplitude difference was about 0.37%. This is small and is explained by the short recording and the filter's start-up behavior. The result supports Y(f) = H(f)X(f).

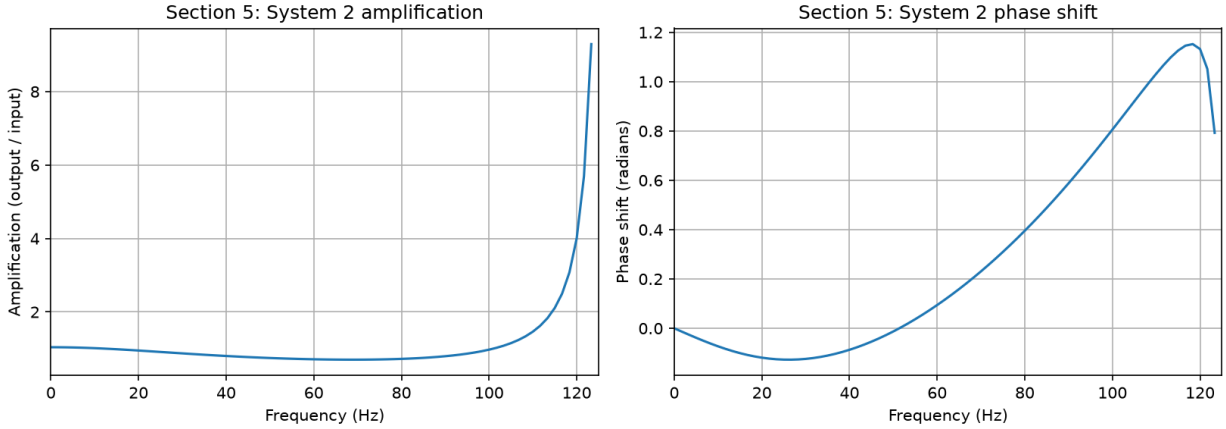


Figure 6. System 2 amplification and phase shift as functions of frequency.

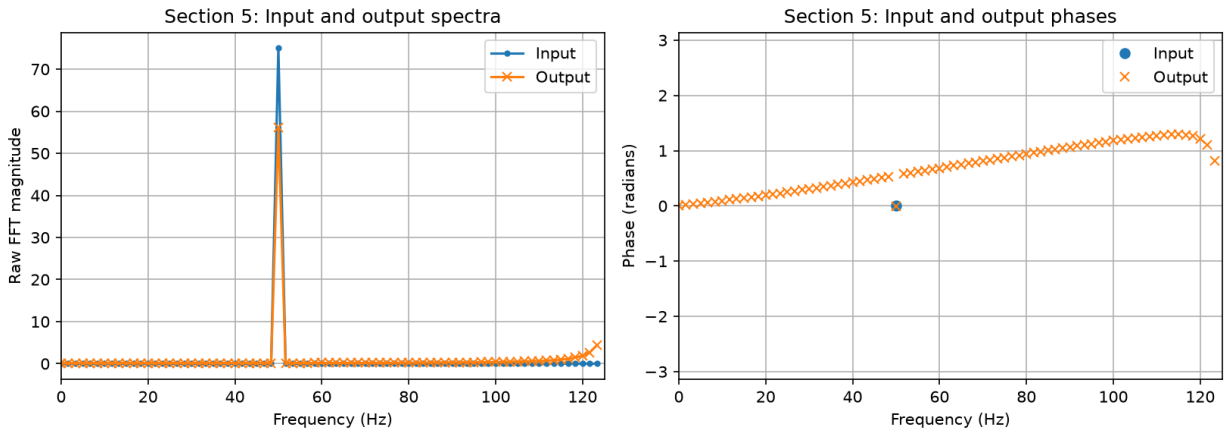


Figure 7. Input and output amplitude and phase spectra for the 50 Hz sinusoidal signal passing through System 2.

3.6 Filtering

Butterworth had a smooth response. Chebyshev Type I had about 3 dB of ripple in the passband but dropped faster after the cutoff. At 300 Hz, their levels were −18.9 dB and −37.4 dB. A low-pass filter keeps lower frequencies, while a high-pass filter keeps higher ones. Raising the cutoff from 200 to 350 Hz passed more high frequencies. Raising the order from 3 to 7 made the cutoff sharper. The phase changed too. At 100 Hz, raising the cutoff from 200 to 350 Hz changed the phase from −1.649 to −0.877 radians. Increasing the order from 3 to 7 changed it from −1.033 to −2.282 radians.

Band-pass and band-stop filters were also made for 150–400 Hz. The band-pass kept this range, while the band-stop reduced it.

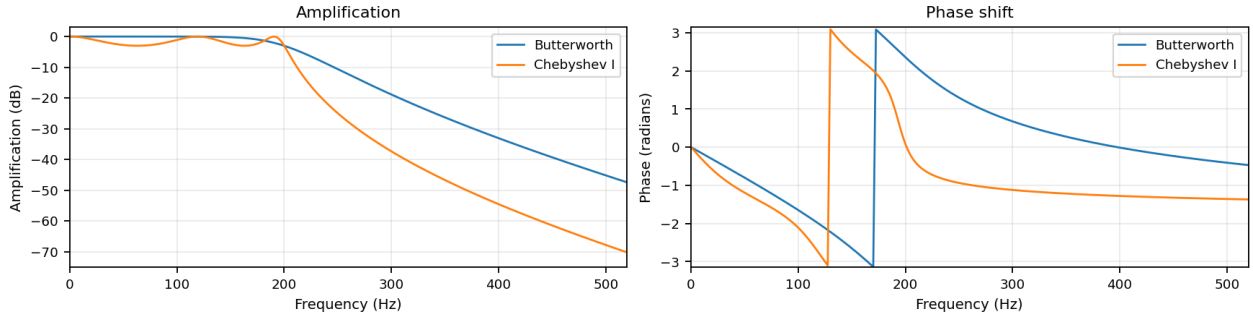


Figure 8. Butterworth and Chebyshev Type I low-pass filters (order 5, cutoff 200 Hz).

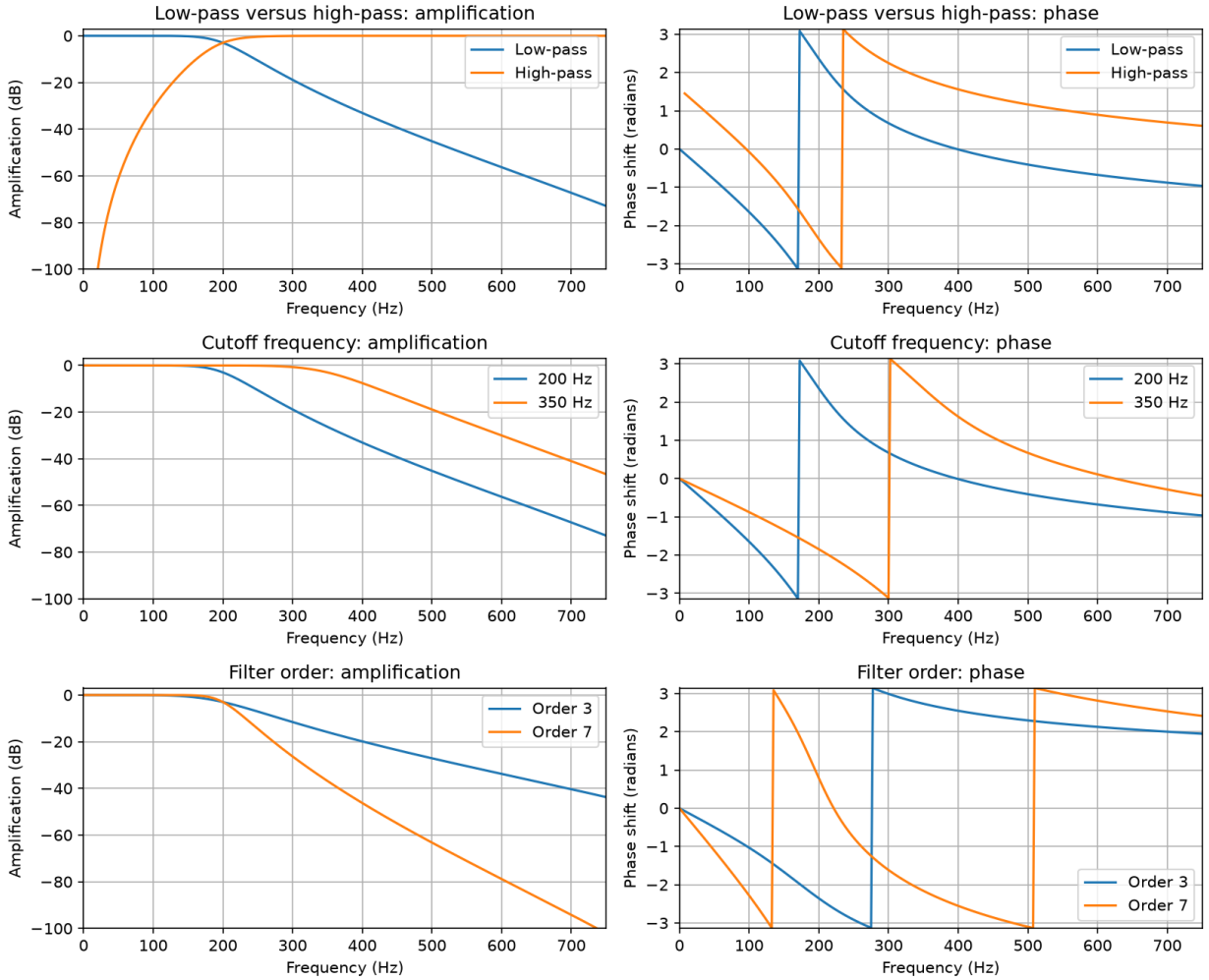


Figure 9. Effect of filter type, cutoff frequency and filter order on the Butterworth filter's amplification and phase response.

3.7 Frequency-time analysis of my voice

I recorded my name. The recording lasted 1.685 seconds at 48,000 Hz. It was split into short parts and the FFT was calculated for each part. This is the short-time Fourier transform, X(t,f). Its squared magnitude gives the power shown in a spectrogram.

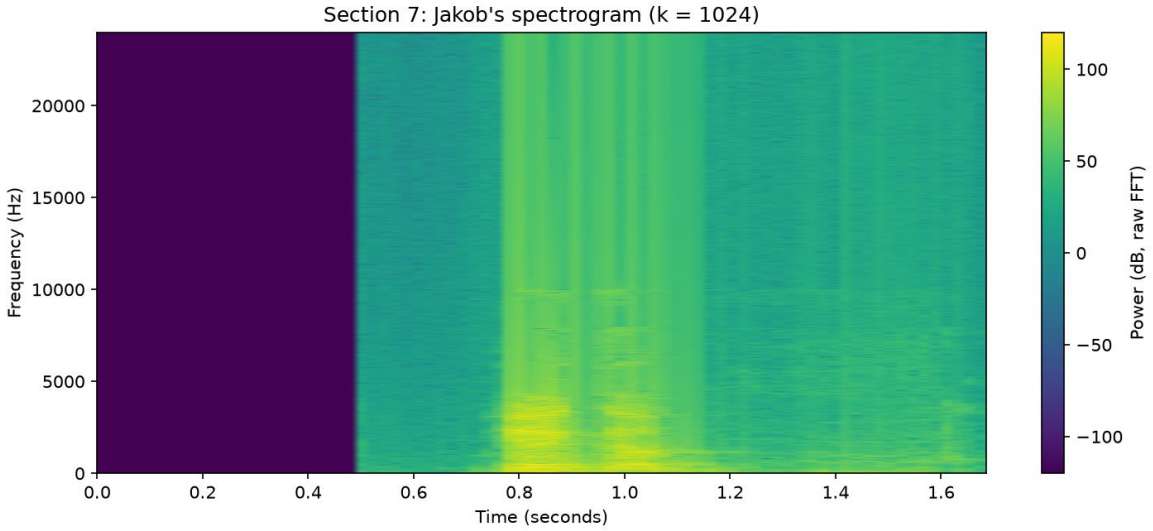


Figure 10. My voice spectrogram, using 1,024 samples per part.

The strongest areas appear around 0.8–1.1 seconds, mainly below 4,000 Hz. Different sound regions are visible, but exact letters or syllables could not be reliably assigned from this plot alone.

The strongest speech activity appears between approximately 0.75 and 1.15 seconds, mainly below 4,000 Hz. The first strong region around 0.77–0.90 seconds appears to correspond to the beginning of "Jakob" ("Ja-"), while the second region around 0.95–1.10 seconds corresponds approximately to the ending ("-kob"). Individual letters cannot be identified with certainty from the spectrogram alone.

Samples per part	Time per part	Frequency spacing
512	10.67 ms	93.75 Hz
1,024	21.33 ms	46.88 Hz
2,048	42.67 ms	23.44 Hz

Smaller parts made quick changes easier to place in time. Larger parts showed frequencies more precisely, but gave less detail about when sounds changed.

4. Discussion

The results show the trade-offs clearly. A window that makes peaks easier to separate in one case will not always be the best choice. Filters can reduce or keep certain frequencies, but their settings also change phase. In my voice recording, the plot showed when the sounds changed, although it was not enough to identify every sound.

5. Conclusion

FFTs were used to find signal frequencies and phases, FIR and IIR systems were compared, and several filters were tested. The spectrogram showed how my voice changed over time. The measurements matched the expected behavior, with only small differences in the finite-length filter test.

Appendix: Code Snippets

These are the main Python commands used in the lab:

```
X = fft(x)                          # Frequency spectrum
phase_x = np.angle(X)              # Phase spectrum
y3 = lfilter(b, a, x2)           # Filter output
b_b, a_b = signal.butter(5, 200, btype="low", fs=2500)
band = [150, 400]
b_bp, a_bp = signal.butter(5, band, btype="bandpass", fs=2500)
b_bs, a_bs = signal.butter(5, band, btype="bandstop", fs=2500)
Xvoice = np.fft.fft(xm_xvoice, axis=1)
power_spectrum = np.abs(Xvoice) ** 2  # Voice power
```

The FFT finds the frequencies in the signal, while np.angle() gives their phases.

lfilter() calculates the system output. The Butterworth commands set up a low-pass filter and filters that keep or remove the 150–400 Hz band.

For the voice recording, axis=1 applies the FFT to each short part. Squaring its magnitude gives the power shown in the spectrogram.

Source: DT081A Lab 2 instructions and course lectures 4–7. Full code and other figures are kept with the lab files.